HiLTL: Hierarchical Temporal Logic Integration for LTL Satisfaction in Reinforcement Learning

Anonymous Authors

***Abstract*—Learning policies that satisfy Linear Temporal Logic (LTL) specifications requires representations of task structure and estimates of long-horizon return. We propose HiLTL, a hierarchical neuro-symbolic framework that augments reach–avoid sequence conditioning with semantically enhanced graph neural networks and a Long–Short Term Fusion architecture. The graph encoder represents automaton node roles, transition types, and guards. Two value pathways use temporal-difference (TD) and Monte Carlo (MC) supervision, and a fixed sigmoid gate varies their weights with automaton proximity. The reported aggregate ZoneEnv comparison gives a +6.4% relative success-rate improvement over the internal baseline. Cross-environment comparisons and component ablations provide further descriptive evidence for the value-learning design on the tested logic-conditioned control tasks.**

***Index Terms*—LTL Specification, Reinforcement Learning, Graph Neural Networks, Neuro-Symbolic AI, Automata, Long–Short Term Fusion.**

I. Introduction

THE integration of formal methods with reinforcement learning provides a principled foundation for developing verifiable and safe autonomous systems [1]. Linear Temporal Logic (LTL) [2] serves as a powerful specification language for expressing complex temporal requirements: safety constraints (e.g., never entering forbidden regions), liveness goals (e.g., eventually reaching targets), and sequential ordering constraints (e.g., visiting locations in a specific order). A central challenge is zero-shot generalization, i.e., training agents on a limited set of LTL specifications and ensuring they can satisfy completely unseen formulas without retraining.

Graph-based automaton representations support reusable task-conditioned policies [3]. DeepLTL [4] provides a strong baseline based on reach–avoid sequence conditioning and automaton-based sequence selection. HiLTL retains this backbone and augments it with semantic graph context and progress-gated value learning.

Figure 1 motivates two design goals. (1) Explicit transition semantics: node roles, progress transitions, non-progress transitions, and epsilon choices provide structural context beyond untyped topology. (2) Progress-conditioned value learning: environmental observations and logical context provide complementary information for estimating discounted return. HiLTL addresses these goals through semantic graph features, TD/MC value supervision, and a fixed progress-dependent fusion rule.

This study proposes HiLTL, shown in Figure 2, with three components: (1) Semantically-Enhanced GNNs (SE-GNN) injecting transition type and node role signals; (2) hierarchical value learning with TD- and MC-supervised pathways; and (3) fixed sigmoid fusion whose weight varies with automaton progress.

II. Related Work

Neuro-symbolic RL integrates formal methods with neural policies [5]. LTL [1, 2] can be translated into LDBAs [6]; our implementation uses Rabinizer [7]. DeepLTL [4] provides a strong baseline using reach–avoid sequence conditioning. Related LTL-RL approaches [8–10] and reward-machine methods [11, 12] expose logical task structure through different interfaces. General graph architectures [13, 14] motivate representation learning, while HiLTL focuses on explicit automaton features and progress-dependent value fusion.

LTL2Action [8] learns formula-conditioned policies for multi-task control. GCRL-LTL [15] combines goal-conditioned reinforcement learning with temporal-logic objectives. Both provide external methodological references; the internal DeepLTL comparison isolates changes within the inherited sequence-conditioning backbone.

Recent advances in GNN expressiveness [16–19] and classic inductive message passing [20] provide complementary perspectives on structural encoding. Graph matching networks [21] further highlight the value of similarity-aware graph representations when structure matters across instances. Hierarchical RL [22–25] learns or specifies temporal abstractions, while LTL automata expose task structure explicitly. Logic-guided RL [26] and neural-network verification [27] address complementary aspects of correctness, while non-Markovian rewards [28] and interpretable policy learning [29] explore related formalisms.

A. Comparative Positioning

Reward-machine approaches [11, 30] expose task structure through reward specifications and automaton state. HiLTL instead focuses on TD- and MC-supervised value pathways and progress-dependent fusion, while retaining the inherited task rewards. Its contribution is this value-learning architecture rather than automaton tracking alone.

Hierarchical RL [23, 24] uses temporal abstractions such as options. HiLTL uses explicit automaton structure to condition value learning. Automaton states provide logical context, and distance to accepting-transition targets provides a structural proximity signal; neither implies that every state transition advances toward acceptance.

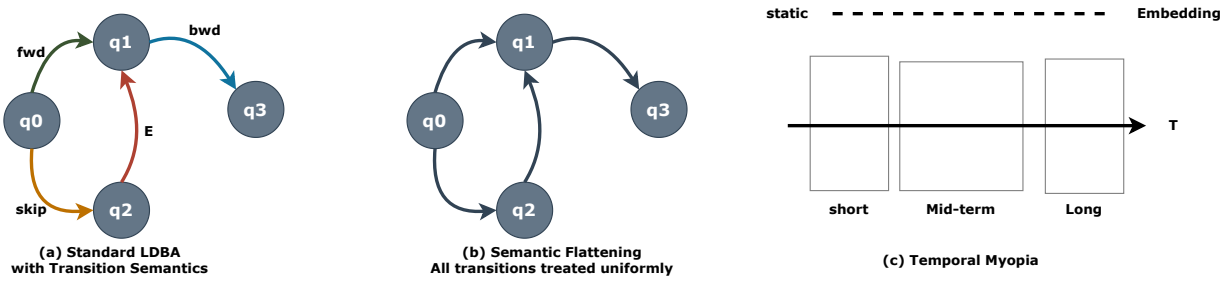


Fig. 1. **Design motivations.** The figure illustrates transition roles and the limitations of omitting task structure and temporal context. The displayed edge labels describe qualitative roles. The operative classifier uses progress, remaining non-epsilon transitions, and epsilon choices, with the skip slot reserved as specified in Eq. 3. TD- and MC-supervised value pathways are combined through progress-dependent fusion.

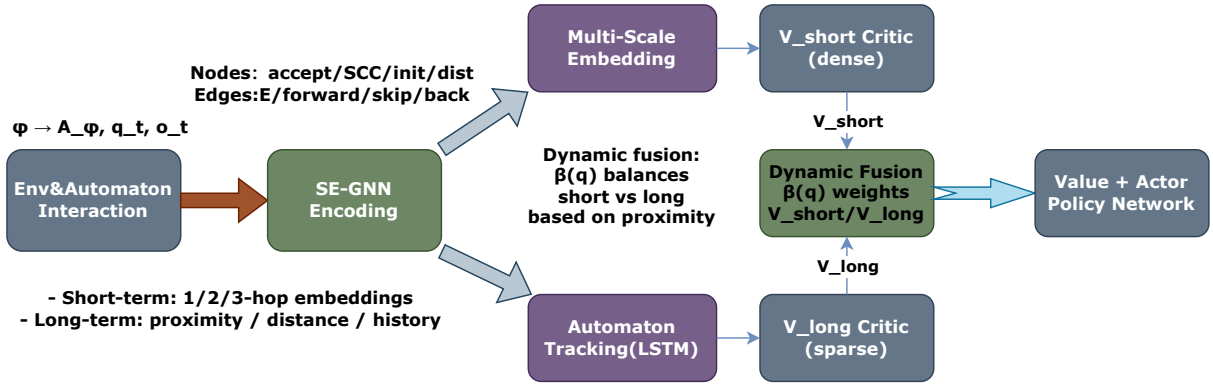


Fig. 2. HiLTL architecture with all representation components shown. SE-GNN encodes automaton features and guards. Multi-layer pooling and history encoding are enabled by configuration. The two value pathways use TD and completed-episode MC supervision, respectively, and a fixed progress-dependent gate fuses their scalar estimates. The actor has a separate output path conditioned on environmental and task representations; the fused value supplies the critic used by PPO. The edge labels and dense/sparse labels in the diagram are schematic.

III. Preliminary

A. Problem Formulation

We seek a policy that maximizes the probability of satisfying an LTL specification φ. Let AP be the atomic propositions and $L : S \to 2^{AP}$ the labeling map. The environment MDP is $\mathcal{M} = (S, A, P, R, \gamma_{\text{RL}})$ and the transition-accepting LDBA is $\mathcal{A}_\varphi = (Q, 2^{AP}, \delta, q_0, F_E)$, with epsilon choices handled explicitly. Here F_E is the set of accepting transitions. The product state is $(s, q) \in S \times Q$. The task-level objective is:

$$\pi^* = \arg\max_{\pi} \Pr[\text{formula } \varphi \text{ is satisfied}] \tag{1}$$

Büchi acceptance requires infinitely many accepting transitions along an infinite run. Training uses the inherited discounted reward objective as a surrogate. We define the return value as:

$$V^\pi(s, q) = \mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \gamma_{\text{RL}}^t r_t \mid s_0 = s, q_0 = q \right] \tag{2}$$

Here $\pi(a \mid s, q)$ is a stochastic policy over product states (s, q), and $P(s' \mid s, a)$ is the environment transition kernel. Rewards r_t include the inherited environment and task rewards on the product trajectory. The value $V^\pi(s, q)$ is the expected discounted return starting from (s, q) under π, with discount $\gamma_{\text{RL}} \in (0, 1)$.

Key notation. Labels satisfy $L(s_t) \in 2^{AP}$, and enabled transitions determine successor logical states, including explicit epsilon choices. Define $Q_F = \{q' : (q, \ell, q') \in F_E\}$ and let $d(q)$ be the shortest directed hop distance from q to Q_F, with $d(q) = \infty$ when no path exists. Set

$$D = \max\left(1, \max_{q : d(q) < \infty} d(q)\right),$$
$$\bar{d}(q) = \frac{\min\{d(q), D\}}{D}, \qquad p(q) = 1 - \bar{d}(q).$$

The inner maximum of an empty set is zero. Unreachable states have $\bar{d}(q) = 1$ and $p(q) = 0$. In subsequent distance notation, $\text{dist}(q, F)$ denotes $d(q)$ and max_dist denotes D; this legacy F refers to Q_F, not F_E. Also $\text{proximity}(q) = p(q)$. These are structural distances rather than physical travel times. Graph and history arguments of value functions are implicit unless needed.

B. Structural Context in LDBAs

We distinguish three effective transition categories:

$$\mathcal{T}_{\text{active}} = \{\tau_{\text{progress}}, \tau_{\text{remaining}}, \tau_\epsilon\}. \tag{3}$$

Epsilon transitions are classified first. A non-epsilon edge (q, ℓ, q') is classified as progress if $d(q') < d(q)$ and as

remaining otherwise. The remaining category includes self-loops, equal-distance edges, and edges leading away from Q_F; it is not restricted to accepting cycles. The four-slot type vector retains a zero skip slot for compatibility. Under shortest-hop distance, a real single edge cannot decrease distance by more than one, so this slot is not presented as an observed multi-hop shortcut.

C. Hierarchical Value Parameterization

We parameterize the fused critic by a convex combination:

$$V(s,q) = (1-\beta(q))V_{\text{short}}(s,q) + \beta(q)V_{\text{long}}(s,q). \quad (4)$$

The two pathways use different inputs and supervision. Their targets and gate are

$$\begin{aligned} y_t^{\text{short}} &= r_t + \gamma_{\text{RL}}(1-m_t)\,\text{stopgrad}[V_{\text{short}}(s_{t+1},q_{t+1})]\,, \\ G_t &= \sum_{k=t}^{T-1} \gamma_{\text{RL}}^{k-t} r_k, \\ \beta(q) &= \sigma(\kappa(p(q)-\theta))\,, \qquad \kappa = 3, \quad \theta = 0.5. \end{aligned}$$

Here m_t is the episode-end mask and T is the number of transitions in a completed episode. The short head is trained against y_t^{short}; the long head predicts G_t from environmental and logical context. Negative rewards are retained. Both heads concern discounted return: one-step TD supervision does not imply a one-step prediction horizon. The long head is a return surrogate rather than a calibrated satisfaction probability. The names short and long distinguish input and supervision pathways rather than an exact additive decomposition of rewards. The fixed gate gives $\beta \in [0.1824, 0.8176]$ approximately for $p \in [0,1]$. Its weight varies with logical progress rather than training time, and progress need not be monotone. Figure 3 illustrates the two pathways.

IV. Methodology

A. Overview of Proposed Approach

As shown in Figure 4, HiLTL combines three elements. First, SE-GNN encodes node roles and transition semantics through edge-conditioned GATv2 attention; its four-slot type vector contains three effective categories and a reserved skip slot. Second, TD- and MC-supervised value pathways are combined by the fixed progress-dependent sigmoid in Eq. 17. Third, when enabled, multi-layer pooling combines representations from one-, two-, and three-hop message-passing depths, providing structural context rather than fixed physical planning horizons.

B. Semantically-Enhanced Graph Neural Networks

1) Graph Structure of Automaton: Node and edge features encode automaton structure. For an infinite trace ρ, $\rho \models \varphi$ denotes LTL satisfaction. A trap state has no accepting continuation. Define

$$h_v(q) = [\mathbb{I}(q \in Q_F), \mathbb{I}(q \in C), \mathbb{I}(q = q_0), \bar{d}(q)], \quad (5)$$

where C is the set of states lying on a directed cycle: its SCC has more than one state or the state has a self-loop. Membership in Q_F is a structural feature and does not replace transition-based Büchi acceptance.

For an edge $e = (q, \ell, q')$, the structural feature vector is

$$\begin{aligned} h_e(e) = [&\mathbb{I}(\text{progress}), \mathbb{I}(\text{remaining}), 0, \\ &\mathbb{I}(\epsilon), \mathbb{I}(q' \in Q_F), \bar{d}(q')]. \end{aligned} \quad (6)$$

Classification follows the epsilon-first rule above. The third type slot is reserved and remains zero; every edge has exactly one active type indicator. The last component is the target's normalized distance, not an absolute distance difference. Transition guards are encoded separately and combined with the structural vector for edge-conditioned attention. No category is assigned a fixed positive or negative attention contribution.

2) Semantically-Conditioned GATv2 Mechanism: An attention score without edge features can be written as follows [31]:

$$e_{ji}^{\text{standard}} = \mathbf{a}^\top \text{LeakyReLU}(W_s \mathbf{h}_j + W_t \mathbf{h}_i). \quad (7)$$

We use the edge-feature interface of GATv2 to incorporate automaton semantics. For an incoming edge $j \to i$,

$$e_{ji} = \mathbf{a}^\top \text{LeakyReLU}(W_s \mathbf{h}_j + W_t \mathbf{h}_i + W_e \mathbf{e}_{ji}), \quad (8)$$

$$\alpha_{ji} = \frac{\exp(e_{ji})}{\sum_{k:(k,i)\in E} \exp(e_{ki})}, \quad (9)$$

where $\mathbf{e}_{ji}$ combines the structural edge features and guard encoding. Edge features allow the network to learn transition-dependent attention weights. A progress indicator does not guarantee a positive contribution or a larger score. For a fixed graph and parameters, graph attention does not change solely because the agent's current state changes; progress conditioning enters through current-node features, optional history, and the value gate.

3) SE-GNN Architecture Details: The SE-GNN consists of three GATv2 layers with residual connections [32] and layer normalization [33], as specified below. Graph transformers [34] and message-passing PDE solvers [35] provide broader architectural context:

$$\mathbf{u}^{(l+1)} = \text{GATv2}(\mathbf{x}^{(l)}, E, \mathbf{e}), \quad (10)$$

$$\mathbf{x}^{(l+1)} = \text{LayerNorm}\big(\mathbf{x}^{(l)} + \text{MLP}_l(\mathbf{u}^{(l+1)})\big). \quad (11)$$

We use three GATv2 layers with four attention heads, residual connections, and LayerNorm. Before graph-level pooling, the layers provide at most one-, two-, and three-hop directed message-passing context. Pooling summarizes node representations, while current-node features condition subsequent prediction. Three hops do not in general give each node global relational information, and graph distance is not a fixed number of physical control steps.

C. MSGNN for Short-Term Features

The short-term value network uses environmental observations and graph context to estimate discounted return under one-step TD supervision. When multi-layer pooling is enabled, it combines embeddings from the SE-GNN layers. We extract embeddings at three aggregation levels,

$$\mathbf{z}_{\text{pool}}^{(l)} = \text{ReadOut}(\mathbf{H}^{(l)}) \quad \text{for } l = 1,2,3 \quad (12)$$

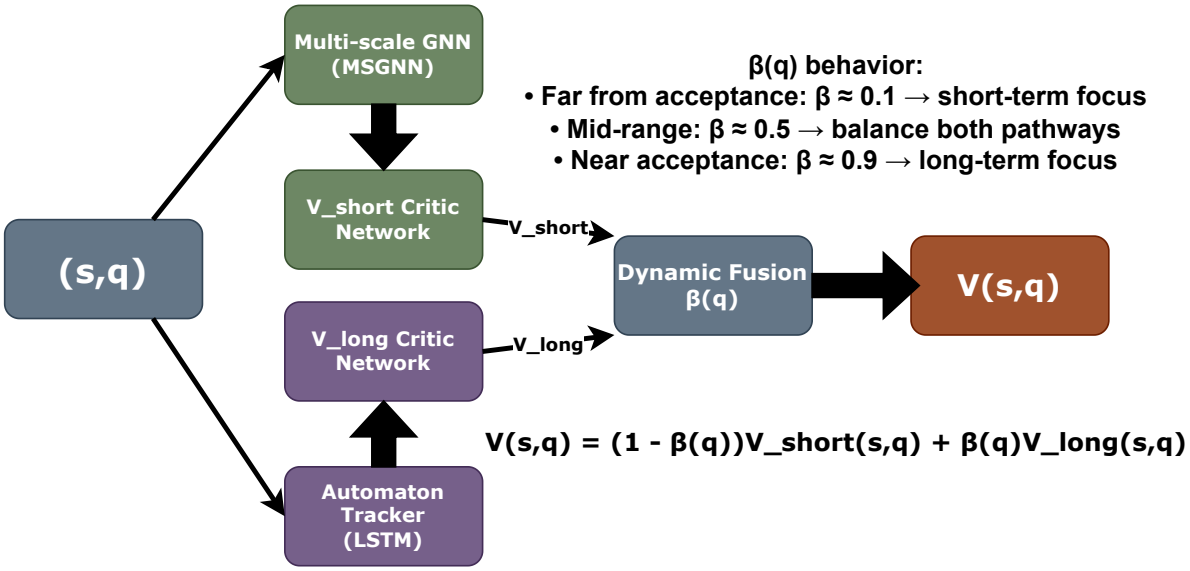


Fig. 3. Fusion overview. Both pathways use environmental and logical context. The short head uses one-step TD supervision, and the long head uses completed-episode discounted returns and optional visited-state history. Multi-layer pooling, when enabled, provides context from three message-passing depths. The fixed sigmoid in Eq. 17 combines the estimates as $V = (1-\beta)V_{\text{short}} + \beta V_{\text{long}}$. The curve in Figure 7 gives endpoint weights of approximately 0.1824 and 0.8176.

where ReadOut is learnable global attention pooling:

$$\text{ReadOut}(\mathbf{H}) = \sum_{i=1}^{n} \text{softmax}_i(\mathbf{w}^T \text{ReLU}(\mathbf{W}_{\text{pool}}\mathbf{h}_i)) \cdot \mathbf{h}_i, \quad (13)$$

where $\mathbf{H} = \{\mathbf{h}_1, \ldots, \mathbf{h}_n\}$ is the node representation set, and $\mathbf{w}$ and $\mathbf{W}_{\text{pool}}$ are learnable parameters. Softmax weights sum to one, giving a graph-level weighted sum. The layers correspond to message-passing depths rather than fixed physical-time horizons. With multi-layer pooling enabled, the representation is

$$\mathbf{z}_{\text{short}} = [\mathbf{z}^{(1)}_{\text{pool}}; \mathbf{z}^{(2)}_{\text{pool}}; \mathbf{z}^{(3)}_{\text{pool}}], \quad (14)$$

and the short-term value function processes the combined representation with the current environment state

$$V_{\text{short}}(s,q) = \text{MLP}_{\text{short}}([\mathbf{s}, \mathbf{z}_{\text{short}}, \mathbf{z}_{\text{GNN}}(q)]), \quad (15)$$

Here $\mathbf{s}$ encodes the environment state, and $\mathbf{z}_{\text{GNN}}(q)$ is the current-node embedding. For configurations without multi-layer pooling, $\mathbf{z}_{\text{short}}$ is replaced by the final-layer pooled representation. The implementation may retain the inherited DeepLTL critic as a residual baseline; this does not change the TD/MC supervision distinction.

D. Automaton Tracking for Long-Term Features

We use three logical-context signals. (1) Directed distance $d(q)$ is computed by reverse multi-source BFS from Q_F, including explicit epsilon edges. (2) History $H_t = [q_0, \ldots, q_t]$ records visited logical states, including repetitions, and is encoded by an LSTM when tracking is enabled. It is reset with the episode and updated on real interaction steps rather than repeated forward passes. (3) Proximity is $p(q) = 1 - \bar{d}(q)$, using the normalization and unreachable-state convention above.

TABLE I
EVALUATION CONFIGURATION MATRIX

Configuration	HV	MSGNN	Track	Options
baseline	×	×	×	×
hv_only	✓	×	×	×
msgnn_only	×	✓	×	×
hv_track	✓	×	✓	×
msgnn_track	×	✓	✓	×
full_no_options	✓	✓	✓	×
full_all	✓	✓	✓	✓

The long head aggregates environmental and logical-context signals:

$$V_{\text{long}}(s_t, q_t) = \text{MLP}_{\text{long}}([\mathbf{s}_t; p(q_t); \mathbf{h}_{q_t}; \text{Enc}(H_t)]). \quad (16)$$

Here $\mathbf{s}_t$ encodes the environment state. The graph and optional history are implicit arguments; without tracking, the history term is omitted. The head predicts completed-episode discounted return. Distance is a structural prior rather than a physical difficulty estimate; higher proximity gives higher gate weight, but neither must increase monotonically within a trajectory.

Configuration scope. HV denotes the dual value heads and fixed progress gate; MSGNN denotes concatenation of pooled representations from multiple layers; Track denotes encoding visited logical-state history. Enhanced variants retain a semantic graph encoder even when MSGNN is disabled. The Options-enabled row is a historical reference outside the six configurations supported by the maintained implementation and is excluded from controlled component-level conclusions.

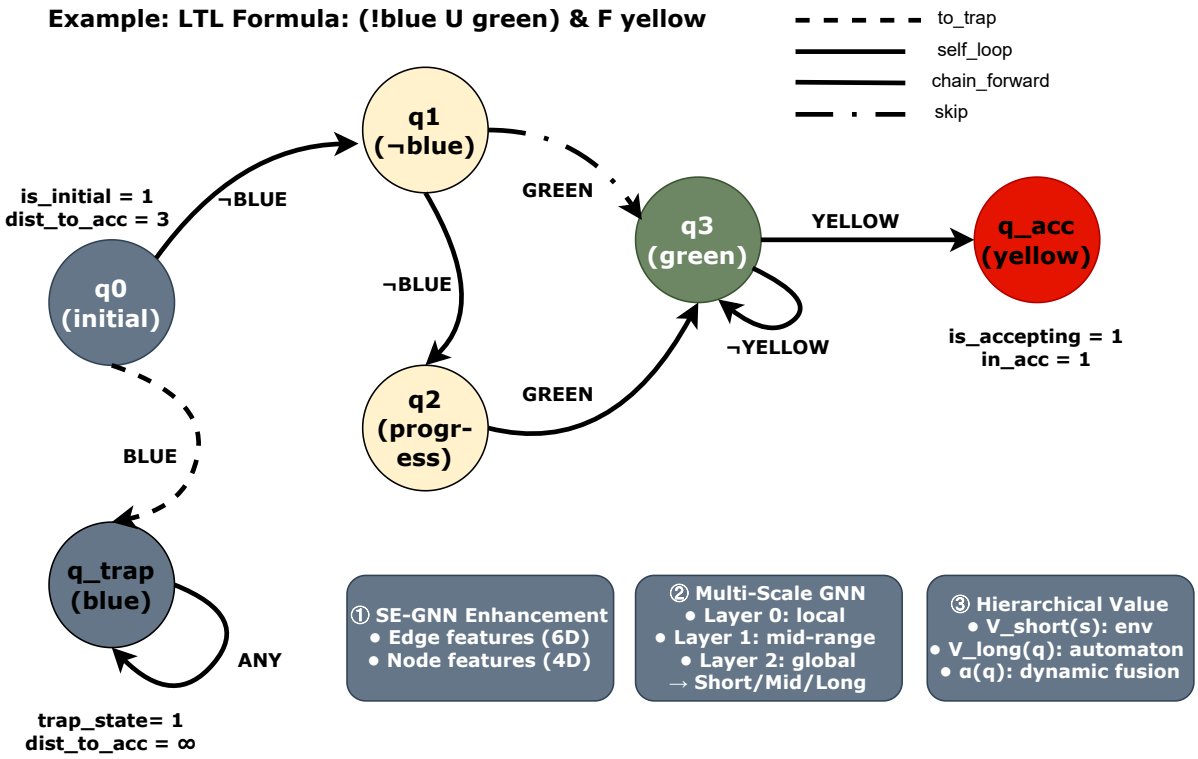


Fig. 4. Conceptual illustration of automaton features motivated by $(\neg\text{blue}\ U\ \text{green}) \wedge F\text{yellow}$. The displayed paths are a partial illustration, not a complete language-equivalent translation of this formula or a full Büchi acceptance structure. Four-dimensional node features encode targets of accepting transitions, cyclic-SCC membership, the initial state, and normalized distance. Six-dimensional edge features are defined in Eq. 6; illustrative edge names do not replace that classifier. Three GNN layers provide one-, two-, and three-hop context, without guaranteeing global information at every node. The value pathways use a fixed progress-dependent gate. Acceptance is defined on transitions.

E. Dynamic Fusion with Adaptive Weighting

The fusion rule adjusts the relative weights of the TD- and MC-supervised heads according to automaton proximity:

$$\beta(q) = \sigma(\kappa(p(q) - \theta)), \qquad \kappa = 3, \quad \theta = 0.5. \tag{17}$$

Here $p(q) = \text{proximity}(q) = 1 - \bar{d}(q)$ and σ is the logistic sigmoid. The gate constants are fixed, with $\beta = 0.5$ at $p = 0.5$. The reward discount is denoted γ_{RL}.

At $p = 0$, $\beta = \sigma(-1.5) \approx 0.1824$; at $p = 0.5$, $\beta = 0.5$; and at $p = 1$, $\beta = \sigma(1.5) \approx 0.8176$. Higher proximity therefore increases the logical-context head's contribution. Smoothness in proximity does not imply continuous actions or policies along discrete automaton transitions.

V. Experiments

A. Experimental Setup

Model parameters and reported protocol. The SE-GNN uses three GATv2 layers with 64-dimensional outputs, four attention heads, LayerNorm, and LeakyReLU ($\alpha = 0.2$). Node and structural edge features have dimensions four and six, respectively; guards are encoded separately. The short and long heads each use two hidden layers, of widths 256 and 128, respectively, followed by a scalar output. Both pathways receive environmental and logical context, and the long pathway optionally receives history. The gate is fixed at $\kappa = 3$ and $\theta = 0.5$. The reported experiments use PPO [36], Adam [37], 1,000,000 training timesteps, minibatches of 64, and learning rate 3×10^{-4}. The reported ZoneEnv discount is $\gamma_{\text{RL}} = 0.998$. Advantages are clipped to $[-1, 1]$ before the actor update; this is separate from PPO's probability-ratio clipping. Evaluation means are reported over seeds 1–5 with 500 episodes per seed. The internal baseline follows the DeepLTL architecture [4]; enhanced configurations add the components specified in Table I.

Environments and tasks. We compare PointLtl2-v0 (ZoneEnv), LetterEnv, and FlatWorld. ZoneEnv uses continuous actions; the evaluated LetterEnv and FlatWorld variants use discrete actions. Table II gives the three finite-task templates for ZoneEnv and FlatWorld. For LetterEnv, blue, green, and yellow are replaced by a, b, and c, respectively. Recurrent and persistence objectives are illustrated by finite trajectory excerpts in Figure 5. The reported configurations are listed in Table I.

External baselines: To broaden comparison beyond internal ablations, we additionally include LTL2Action [8] and GCRL-LTL [15] in the main performance table and cross-environment analyses. External baseline values are retained from the original DeepLTL study and are reported as historical reference results; internal comparisons use our evaluation pipeline.

TABLE II
TASK EVALUATION SET

Property	Value
Task φ_1	
Formula	F blue $\wedge$ F green
Difficulty	Easy
LDBA Size	5 states
Type	Conjunctive reachability
Task φ_2	
Formula	$(F \text{ blue}) \wedge (F \text{ green}) \wedge (F \text{ yellow})$
Difficulty	Medium
LDBA Size	12 states
Type	Aggregation
Task φ_3	
Formula	$(\neg \text{ blue } U \text{ green}) \wedge F$ yellow
Difficulty	Hard
LDBA Size	18 states
Type	Constraint

Automaton counts. State counts in Table II refer to the reported experiments and depend on translation and simplification. The current maintained code produces 4, 8, and 5 states for the corresponding templates after legal-valuation pruning; these are different conversion artifacts, not replacements for the historical counts.

Evaluation metrics. SR denotes the proportion of completed test episodes satisfying a finite-task objective. Reported evaluation means use seeds 1–5 and 500 episodes per seed. Secondary metrics are mean completion steps and cumulative reward. Relative SR gain is $100(\mathrm{SR}_{\mathrm{method}}/\mathrm{SR}_{\mathrm{reference}} - 1)\%$; absolute SR differences are reported in percentage points (pp). Derived quantities are calculated from displayed values before rounding. The comparisons below are descriptive; no inferential significance claim is made.

TABLE III
HV-RELATED AGGREGATE CONFIGURATION PERFORMANCE

Configuration	Avg SR	Relative gain
baseline	0.933	—
hv_only	0.975	+4.5%
hv_track	**0.993**	**+6.4%**
full_all	0.963	+3.2%

B. Overall Performance

The reported aggregate comparison includes seven configurations, with full_all retained as a historical Options-enabled reference. Figure 5 illustrates behavior under recurrent and persistence objectives. In the aggregate comparison, hv_track achieves 99.3% SR, corresponding to a +6.4% relative gain over the internal baseline; hv_only and msgnn_only achieve +4.5% and +2.3%, respectively. These results motivate investigation of hierarchical value learning. Evaluation on larger automata remains future work.

Table VII summarizes aggregate results. Among the reported internal variants, hv_track has the highest average SR (0.993) and average reward (0.663), with 235.5 mean steps. The corresponding baseline values are 0.933, 0.618, and 231.3.

TABLE IV
SCOPE OF THE SELECTED BASELINE FAMILIES

Method	Task interface and comparison role
LTL2Action	Formula-conditioned policy; historical reference
GCRL-LTL	Goal-conditioned composition; historical reference
DeepLTL	Reach–avoid sequences and automaton-based selection; internal baseline
HiLTL	Semantic graph context and progress-gated value learning

The hv_only and msgnn_only variants improve aggregate SR by +4.5% and +2.3%, respectively, while full_no_options is 0.4% below baseline. These means describe configuration-level differences and do not identify a unique optimization mechanism.

The aggregate results favor hierarchical value learning: hv_track exceeds hv_only by 1.8 percentage points (0.993 versus 0.975), or approximately 1.85% relative to hv_only. Component-removal results are reported in Section V-H2. Larger-automaton evaluation is discussed in Appendix B.

C. External Baseline Coverage

To strengthen baseline completeness, we complement internal ablations with two representative external families: structure-conditioned policy learning (LTL2Action) and compositional goal-conditioned planning (GCRL-LTL). Table IV summarizes methodological coverage and clarifies why both are retained in addition to our internal variants. These historical references broaden methodological coverage; controlled architectural comparisons are provided by the internal variants.

The reported aggregate SR of hv_track exceeds the historical LTL2Action and GCRL-LTL reference values by 27.6 and 12.0 percentage points, respectively (0.993 versus 0.717 and 0.873). These are descriptive differences. Because the external methods were not rerun under a matched protocol, these comparisons do not isolate the effect of HiLTL's components.

D. Cross-Environment Performance

We provide a cross-environment view over ZoneEnv, LetterEnv, and FlatWorld. Table V reports averaged SR over three representative finite-horizon formulas per environment.

Both hv_only and hv_track outperform the internal baseline across the three environments. The hv_track variant has the higher SR on ZoneEnv and LetterEnv, while hv_only is slightly higher on FlatWorld (0.978 versus 0.976). Averaging the displayed environment-level SRs gives 0.984 for hv_track, compared with 0.943 for the internal baseline, 0.743 for LTL2Action, and 0.877 for GCRL-LTL. These comparisons show positive mean performance differences across the tested environments.

E. Recurrent and Persistence Objectives

Figure 5 provides finite trajectory excerpts for $GF\,\mathrm{blue} \wedge GF\,\mathrm{yellow}$ and $FG\,\mathrm{green}$. The first objective requires repeated visits to both colors; the second requires eventual permanent

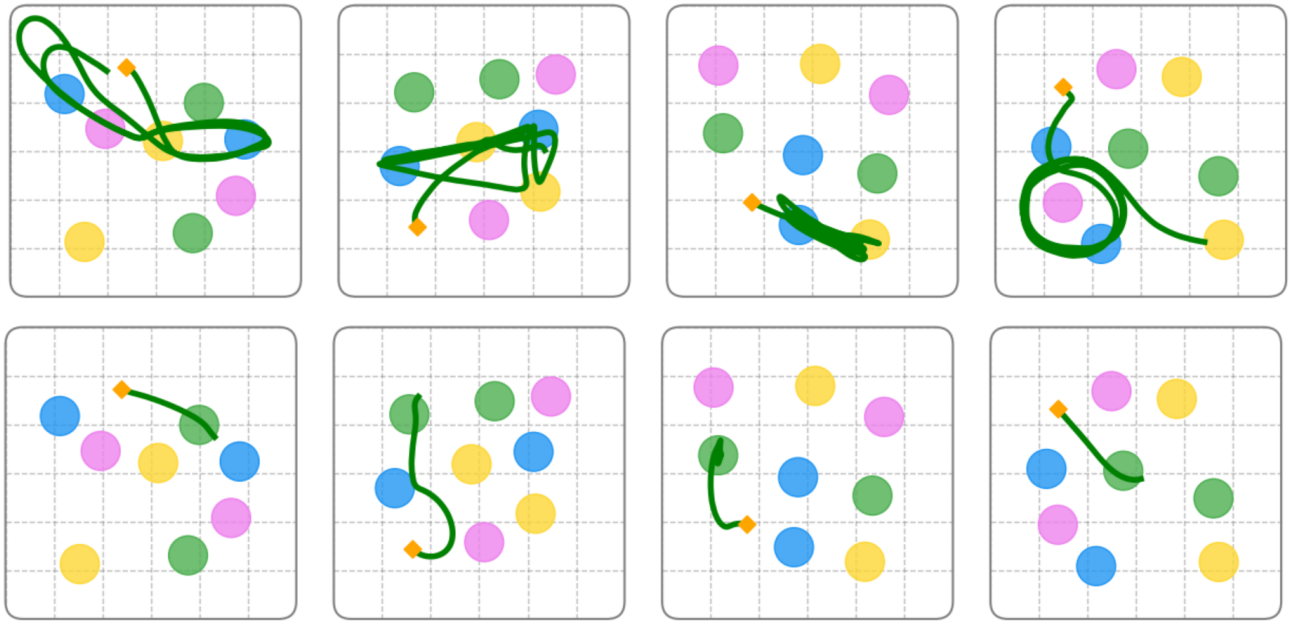


Fig. 5. ZoneEnv visualization examples under two typical LTL objectives, shown as two rows spanning both columns. Top: GF blue $\wedge$ GF yellow, with repeated visits to blue and yellow regions. Bottom: FG green, with residence in the green region over the displayed window. These finite trajectories illustrate behavior and do not establish infinite-horizon satisfaction.

TABLE V
CROSS-ENVIRONMENT SUCCESS RATE COMPARISON (FINITE-HORIZON)

Method	ZoneEnv	LetterEnv	FlatWorld	Avg SR
LTL2Action	0.717	0.622	0.890	0.743
GCRL-LTL	0.873	0.940	0.818	0.877
baseline (internal)	0.933	0.941	0.956	0.943
hv_only (internal)	0.975	0.971	**0.978**	0.975
hv_track (internal)	**0.993**	**0.984**	0.976	**0.984**

residence in green. These excerpts illustrate observed behavior within a finite window. They do not establish infinite-trace satisfaction or an infinite-horizon success probability.

F. Impact of Task Difficulty on Hierarchical Value Networks

The task-level annotations in Figure 6 report positive gains for hv_track on the three displayed tasks: +1.2% on φ_1, +2.9% on φ_2, and +4.3% on φ_3. The largest displayed gain occurs on constrained reachability. Formula structure and automaton size both vary across these examples, so this comparison does not isolate an effect of automaton size. Steps and rewards are reported separately in Table VIII.

Table III reports aggregate configuration comparisons. The component-removal study in Section V-H2 separately examines the contribution of the value network, tracking, and fusion on φ_3.

The aggregate relative SR gains are +4.5% for hv_only and +6.4% for hv_track. The full_all reference gives +3.2%, while full_no_options is 0.4% below baseline. Since full_all also enables Options, its comparison with hv_track does not isolate MSGNN. The highest reported aggregate SR is obtained by hv_track. The results motivate further controlled comparisons of component interactions rather than establishing a general interference mechanism.

G. Hierarchical Value Effectiveness

Three design choices motivate the hierarchical critic. First, the two heads use TD and MC supervision for discounted return; the reported aggregate SR rises from 93.3% for baseline to 99.3% for hv_track. Second, the long head combines environmental state, logical state, and optional history; the component-removal comparison reports a 1.0 percentage-point reduction without tracking. Third, the fixed gate increases from approximately 0.1824 at $p = 0$ to 0.8176 at $p = 1$, with $\beta = 0.5$ at $p = 0.5$. Removing dynamic fusion reduces the reported single-task SR by 0.5 percentage points. These observations are consistent with a benefit from the tested design choices, but do not establish a unique causal explanation.

H. Detailed Metrics and Component Ablations

1) Performance Metrics Details: We analyze detailed metrics (steps and rewards) across configurations to understand the quality-versus-efficiency trade-off. Table VIII reports average steps and rewards for three HV-related configurations.

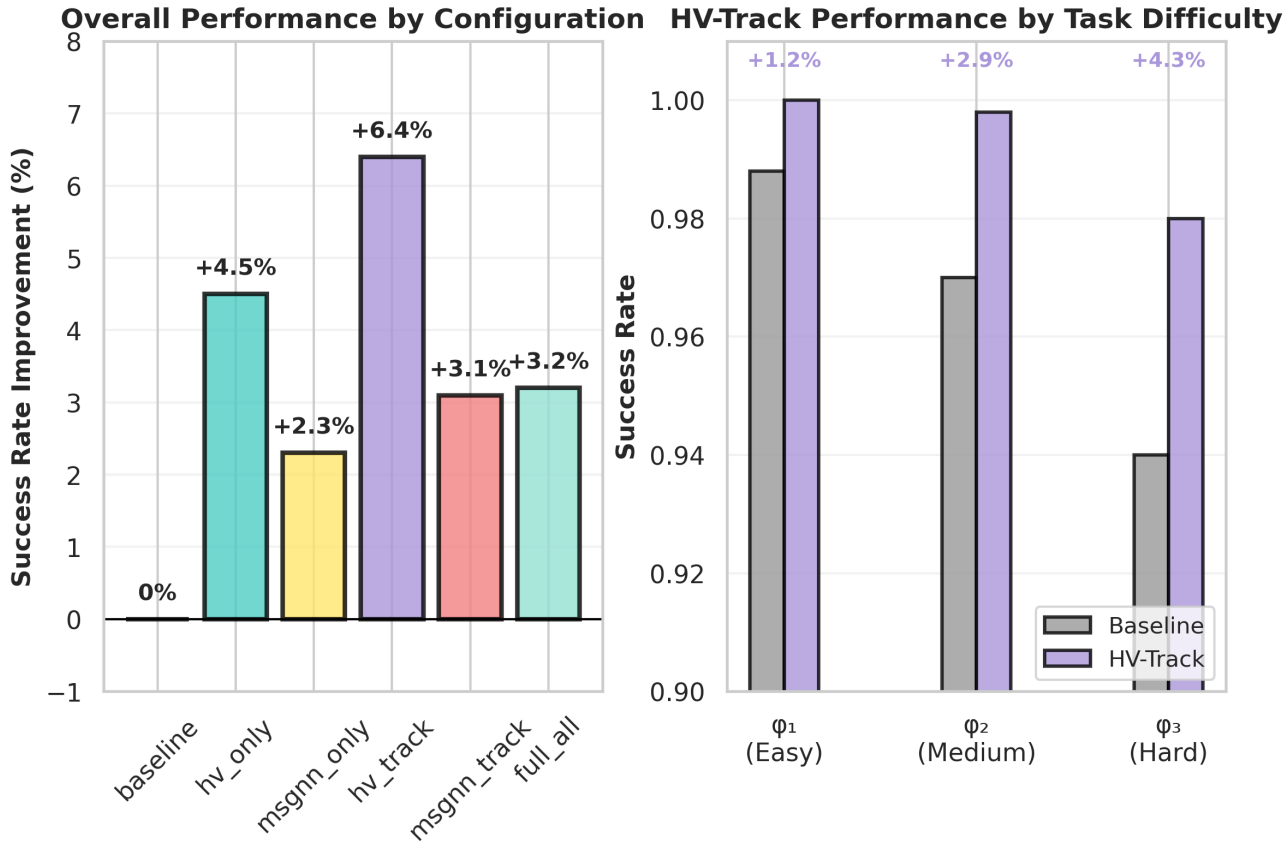


Fig. 6. Reported SR comparisons. **Left:** relative aggregate gains over baseline are +4.5% (hv_only), +2.3% (msgnn_only), +6.4% (hv_track), +3.1% (msgnn_track), and +3.2% (full_all). **Right:** task-level annotations for hv_track are +1.2%, +2.9%, and +4.3%. The two panels are reported separately; the right-panel annotations are not averaged to obtain the left-panel aggregate.

TABLE VI
SUMMARY OF COMPONENT CONTRIBUTIONS (QUANTITATIVE DETAILS IN REFERENCED TABLES)

Component	**Key Result (Source)**
SE-GNN	Semantic feature design (Eqs. 5–6)
Hierarchical Value	Main configuration gains (Table III)
Dynamic Fusion	Component ablation (Table IX)
Automaton Tracking	Component ablation (Table IX)
Task Difficulty	Comparison on the tested tasks (Figure 6)

The reported aggregate average reward increases from 0.618 to 0.663, a relative improvement of approximately 7.3%. The corresponding mean step counts are 231.3 and 235.5, an increase of approximately 1.8%. On φ_3, the reported mean step count increases from 201.9 to 207.4, a difference of 5.5 steps. These means describe a reward–step trade-off; they do not establish a reduction in trajectory variance or identify the cause of individual successes.

2) Component Ablation Study: To isolate the contribution of each architectural component, we conduct systematic ablation experiments based on the best-performing hv_track configuration.

The component-removal results are reported for φ_3 using the stated PPO budget and learning rate. Each row removes the named mechanism from hv_track. These comparisons are separate from the aggregate configuration comparison.

The component-removal results in Table IX show SR reductions of 1.8, 1.0, and 0.5 percentage points when the HV network, automaton tracking, and dynamic fusion are removed, respectively. The largest reduction in this comparison follows removal of the HV network. These single-task component-removal effects are distinct from the aggregate configuration gains in Table III.

VI. CONCLUSION

HiLTL combines semantic automaton representations, TD- and MC-supervised value pathways, and progress-dependent fusion. The reported aggregate ZoneEnv comparison gives a +6.4% relative SR gain for hv_track over the internal baseline. Cross-environment means and the single-task component-removal study provide complementary descriptive comparisons. The trajectory illustrations demonstrate behavior under recurrent and persistence objectives within finite windows. The results motivate hierarchical value learning for the tested logic-conditioned control tasks; they do not provide a formal guarantee of satisfaction or safety.

The best reported aggregate result is obtained by hv_track, while the additional value of multi-layer pooling depends on configuration. Future work should test larger automata, specify

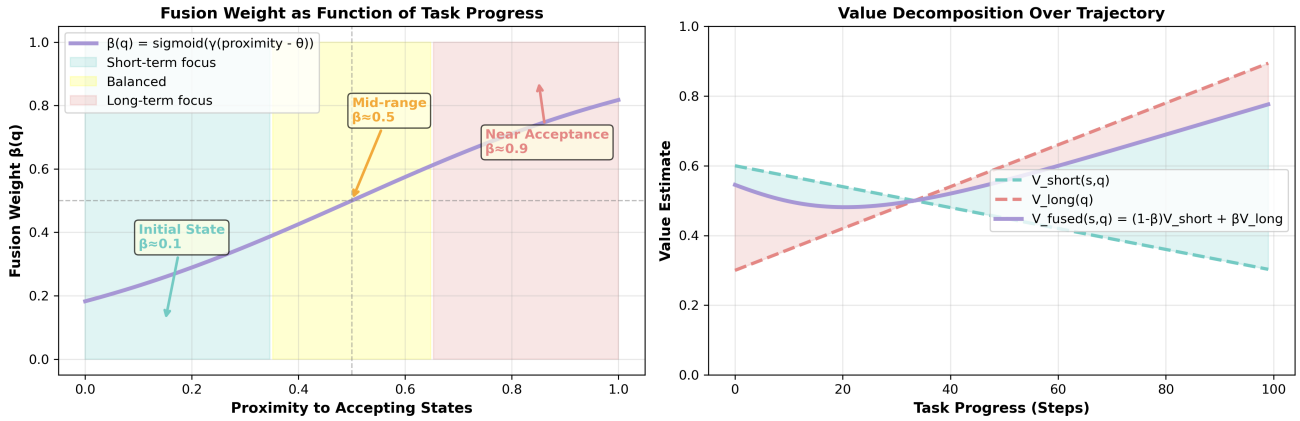


Fig. 7. Illustration of progress-dependent fusion. **Left:** the plotted sigmoid corresponds to $\beta = \sigma(3(p - 0.5))$, with values approximately 0.1824, 0.5, and 0.8176 at $p = 0$, 0.5, and 1. The slope symbol γ in the image denotes κ, separate from the RL discount. **Right:** illustrative value pathways and their convex fusion over a trajectory; $V_{\text{long}}(q)$ suppresses environmental and optional history arguments. The value curves illustrate the fusion rule and are not an empirical convergence analysis.

TABLE VII

REPORTED AGGREGATE PERFORMANCE ON POINTLTL2-V0. INTERNAL RESULTS USE 500 EPISODES PER SEED, AVERAGED OVER SEEDS 1–5. EXTERNAL BASELINE VALUES ARE HISTORICAL REFERENCES; THEIR ORIGINAL TASK SUITE AND PROTOCOL DIFFER FROM THE INTERNAL COMPARISON. RELATIVE GAINS ARE COMPUTED FROM THE DISPLAYED AGGREGATE SRS AND ROUNDED TO ONE DECIMAL PLACE.

Model	Avg SR	Relative gain	Avg Steps	Avg Reward
LTL2Action	0.717	—	232.2	0.589
GCRL-LTL	0.873	—	233.5	0.595
baseline	0.933	—	231.3	0.618
hv_only	0.975	+4.5%	236.8	0.652
msgnn_only	0.954	+2.3%	240.3	0.636
hv_track	**0.993**	**+6.4%**	235.5	**0.663**
msgnn_track	0.962	+3.1%	241.8	0.630
full_no_options	0.929	-0.4%	245.2	0.602
full_all	0.963	+3.2%	243.1	0.648

TABLE VIII

HV IMPACT ON AVERAGE STEPS AND REWARDS

Configuration	φ_1 Steps	φ_2 Steps	φ_3 Steps	Avg Steps	φ_3 Reward	Avg Reward
baseline	197.5	294.5	201.9	231.3	0.635	0.618
hv_only	209.4	300.2	200.9	236.8	0.652	0.652
hv_track	196.8	302.3	207.4	235.5	0.654	0.663

TABLE IX

ABLATION STUDY (BASED ON HV_TRACK)

Removed Component	φ_3 SR	Δ SR (pp)	Avg Reward
Complete (hv_track)	0.980	—	0.654
- HV network	0.962	-1.8	0.635
- Automaton Tracking	0.970	-1.0	0.644
- Dynamic fusion ($\beta(q)$)	0.975	-0.5	0.650

controlled component interventions, and measure computational cost under matched training budgets. These evaluations are needed before making general claims about scalability or architectural interactions.

APPENDIX A
ALGORITHM SPECIFICATIONS

This appendix provides detailed algorithmic specifications of HiLTL's three core components: SE-GNN construction, hierarchical value decomposition, and the complete training procedure.

A. SE-GNN: Semantically-Enhanced Graph Construction

Algorithm 1 describes how we construct the enhanced automaton graph with semantic features.

Edge Type Classification. For $e = (q, \ell, q')$, the classifier uses the following ordered rules:

- `epsilon`: the edge is an epsilon choice.
- `progress`: a non-epsilon edge with $d(q') < d(q)$.
- `remaining`: any other non-epsilon edge, including self-loops and equal-distance edges.

Algorithm 1 SE-GNN Graph Construction

Require: LDBA $\mathcal{A} = (Q, 2^{AP}, \delta, q_0, F_E)$
Ensure: Graph (V, E, X_v, X_e) and transition guards
$E \leftarrow$ the directed transitions of $\mathcal{A}$
$Q_F \leftarrow \{q' : (q, \ell, q') \in F_E\}$
Compute SCCs of the directed graph
$d \leftarrow$ reverse multi-source BFS from Q_F; unreachable distances are ∞
$D \leftarrow \max(1, \max_{q:d(q)<\infty} d(q))$; inner empty maximum is zero
$\bar{d}(q) \leftarrow \min\{d(q), D\}/D$ for all q
for each state $q \in Q$ **do**
  $is_target \leftarrow \mathbb{I}(q \in Q_F)$
  $has_cycle \leftarrow \mathbb{I}$(SCC size > 1 or a self-loop exists)
  $is_initial \leftarrow \mathbb{I}(q = q_0)$
  $X_v[q] \leftarrow [is_target, has_cycle, is_initial, \bar{d}(q)]$
end for
for each edge $e = (q, \ell, q') \in E$ **do**
  $edge_type \leftarrow$ ClassifyTransition(e, d), testing epsilon first
  $type_onehot \leftarrow$ OneHot$(edge_type, 4)$
  Use order [progress, remaining, reserved, epsilon]
  $X_e[e] \leftarrow [type_onehot, \mathbb{I}(q' \in Q_F), \bar{d}(q')]$
  Retain the guard for separate encoding
end for
return $(V = Q, E, X_v, X_e)$ and guards

The skip slot is reserved and zero; no real edge is classified by a shortest-hop decrease greater than one.

B. Hierarchical Value Decomposition with Dynamic Fusion

Algorithm 2 presents the hierarchical value computation during policy evaluation.

Algorithm 2 Hierarchical Value Estimation

Require: State s_t, logical state q_t, graph embeddings, optional history H_t
Require: Value networks $V_{\text{short}}, V_{\text{long}}$; fixed $\kappa = 3, \theta = 0.5$
Ensure: Fused value and pathway estimates
$p \leftarrow 1 - \bar{d}(q_t)$
$v_s \leftarrow V_{\text{short}}(s_t, q_t)$ using the short-head representation
if tracking is enabled **then**
  $h_H \leftarrow \text{LSTM}(\mathbf{h}_{q_0}, \ldots, \mathbf{h}_{q_t})$
  $v_l \leftarrow \text{MLP}_{\text{long}}([\mathbf{s}_t; p; \mathbf{h}_{q_t}; h_H])$
else
  $v_l \leftarrow \text{MLP}_{\text{long}}([\mathbf{s}_t; p; \mathbf{h}_{q_t}])$
end if
$\beta \leftarrow \sigma(\kappa(p - \theta))$
$V_{\text{total}} \leftarrow (1 - \beta)v_s + \beta v_l$
return $V_{\text{total}}, \{v_s, v_l, \beta\}$

Intuition: The MC-supervised long head receives weight approximately 0.1824 at zero proximity, 0.5 at medium proximity, and 0.8176 at unit proximity. The rule is fixed in parameter space and varies with logical state. Proximity need not increase monotonically within an episode.

C. Complete Training Procedure

Algorithm 3 summarizes training with a reach–avoid curriculum and vectorized PPO rollouts. The logical context C_t includes the sampled sequence monitor, current logical state, and optional visited-state history. At evaluation, the test formula is translated to an LDBA and the inherited sequence-selection procedure supplies policy conditioning. Training monitors and test-formula automata therefore have different roles. The trainable parameters are denoted Θ, distinct from the fixed gate threshold θ.

The auxiliary losses apply when HV is enabled. The long head receives environmental and logical context, with optional history, as specified in Algorithm 2. Both heads predict discounted return. Here $m_t = 1$ at a defined episode boundary, including the finite task horizon, and $m_t = 0$ otherwise; rollout boundaries alone do not end an episode. Incomplete episodes are not assigned zero MC targets. Completed episodes may span policy updates, so their MC returns need not have been collected under one frozen policy. The gate constants are fixed and are distinct from the RL reward discount.

APPENDIX B
FUTURE DIRECTIONS FOR MSGNN

The configuration comparisons in Section V-H motivate the following research directions. They are proposed evaluations, not completed experiments or quantitative performance forecasts.

A. Direction 1: Larger Automata

Evaluate richer proposition sets, nested formulas, and safety–liveness combinations. Record automaton size and structure after a fixed translation and pruning protocol. Graph encoders [13, 38] provide candidate architectures, while probabilistic modeling [39] offers complementary perspectives. Their performance on these tasks should be measured under matched budgets; graph size alone does not determine the required message-passing depth.

B. Direction 2: Architecture and Cost

Compare GNN depth, attention heads, hidden dimensions, and pooling operators with multiple independent training seeds. Report parameter counts, training time, inference latency, and performance. Offline meta-RL [40] is a separate possible adaptation direction and is not evaluated here.

C. Direction 3: Semantic Features

Transition guards and acceptance-target distances are already represented. Future ablations should test richer interactions between guard semantics and graph structure, recurrent-acceptance information, and logical implication features. Such comparisons should separate feature contributions from additional model capacity.

Algorithm 3 HiLTL Training with TD/MC Value Supervision

Require: Environment, reach–avoid curriculum, rollout budget
Require: $\lambda_{\text{short}} = 1$, $\lambda_{\text{long}} = 0.3$
Ensure: Policy and value-network parameters Θ
Initialize policy, graph encoder, and enabled value heads
Set fixed gate constants $\kappa = 3$, $\theta = 0.5$
Initialize parallel environments and per-environment episode buffers
while the training budget remains **do**
  Sample curriculum tasks at environment resets
  Construct their sequence monitors and semantic graphs
  Collect a vectorized rollout with the current policy
  Use environment, sequence, and enabled graph context for actions
  Handle enabled epsilon choices through the logical wrapper
  Store rewards, actions, old log probabilities, values, and context
  Update logical history on interaction steps; reset at episode boundaries
  Compute PPO advantages and returns using the fused critic
  Clip actor advantages to $[-1, 1]$ under the reported protocol
  Compute fixed short-head targets with collection-time parameters:
  $y_t \leftarrow r_t + \gamma_{\text{RL}}(1 - m_t)V_{\text{short,old}}(s_{t+1}, q_{t+1})$
  For completed episodes, compute G_t by discounted backward accumulation
  Retain unfinished episode buffers for subsequent collection
  for each PPO epoch and minibatch **do**
    Recompute trainable graph and history representations
    Evaluate the policy and enabled value heads
    $\mathcal{L}_s \leftarrow \text{mean}_t(V_{\text{short},t} - y_t)^2$
    $\mathcal{L}_l \leftarrow \text{mean}_t(V_{\text{long},t} - G_t)^2$
    Use $\mathcal{L}_l = 0$ if no completed-episode examples exist
    Compute clipped PPO actor and fused-critic losses
    $\mathcal{L} \leftarrow \mathcal{L}_{\text{PPO}} + \lambda_{\text{short}}\mathcal{L}_s + \lambda_{\text{long}}\mathcal{L}_l$
    Update Θ with Adam; invalidate cached embeddings
  end for
  Discard consumed completed-episode MC examples
end while
return Θ

D. Direction 4: Co-Design with Value Learning

Investigate how representations at different message-passing depths should condition the value pathways. Three-hop context is not necessarily global context. Vary MSGNN, Options, and value-pathway routing separately: full_all changes more than one component relative to hv_track, so their performance difference cannot identify a unique interaction. No numerical improvement is predicted before these comparisons are performed.

REFERENCES

[1] C. Baier and J.-P. Katoen, *Principles of Model Checking*. MIT Press, 2008.

[2] A. Pnueli, "The temporal logic of programs," in *18th Annual Symposium on Foundations of Computer Science*, 1977, pp. 46–57. [Online]. Available: https://doi.org/10.1109/sfcs.1977.32

[3] B. Yalcinkaya, N. Lauffer, M. Vazquez-Chanlatte, and S. Seshia, "Compositional Automata Embeddings for Goal-Conditioned Reinforcement Learning," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 72 933–72 963. [Online]. Available: https://papers.nips.cc/paper_files/paper/2024/hash/858fc542b70d3b39067f7d3b1cd77635-Abstract-Conference.html

[4] M. Jackermeier and A. Abate, "DeepLTL: Learning to Efficiently Satisfy Complex LTL Specifications for Multi-Task RL," in *International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=9pW2J49flQ

[5] K. Jothimurugan, S. Bansal, O. Bastani, and R. Alur, "Compositional Reinforcement Learning from Logical Specifications," in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 10 026–10 039. [Online]. Available: https://papers.nips.cc/paper_files/paper/2021/hash/531db99cb00833bcd414459069dc7387-Abstract.html

[6] S. Sickert, J. Esparza, S. Jaax, and J. Křetínský, "Limit-Deterministic Büchi Automata for Linear Temporal Logic," in *Computer Aided Verification*, 2016, pp. 312–332. [Online]. Available: https://doi.org/10.1007/978-3-319-41540-6_17

[7] J. Křetínský, T. Meggendorfer, S. Sickert, and C. Ziegler, "Rabinizer 4: From LTL to Your Favourite Deterministic Automaton," in *Computer Aided Verification*, 2018, pp. 567–577. [Online]. Available: https://doi.org/10.1007/978-3-319-96145-3_30

[8] P. Vaezipoor, A. C. Li, R. A. Toro Icarte, and S. A. McIlraith, "LTL2Action: Generalizing LTL Instructions for Multi-Task RL," in *International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 139. PMLR, 2021, pp. 10 497–10 508. [Online]. Available: https://proceedings.mlr.press/v139/vaezipoor21a.html

[9] E. M. Hahn, M. Perez, S. Schewe, F. Somenzi, A. Trivedi, and D. Wojtczak, "Model-Free Reinforcement Learning for Lexicographic Omega-Regular Objectives," in *Formal Methods*, 2021, pp. 142–159. [Online]. Available: https://doi.org/10.1007/978-3-030-90870-6_8

[10] A. K. Bozkurt, Y. Wang, M. M. Zavlanos, and M. Pajic, "Control Synthesis from Linear Temporal Logic Specifications using Model-Free Reinforcement Learning," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 10 349–10 355. [Online]. Available: https://doi.org/10.1109/icra40945.2020.9196796

[11] R. Toro Icarte, T. Q. Klassen, R. Valenzano, and S. A. McIlraith, "Reward Machines: Exploiting Reward

Function Structure in Reinforcement Learning," *Journal of Artificial Intelligence Research*, vol. 73, pp. 173–208, 2022. [Online]. Available: https://doi.org/10.1613/jair.1.12440

[12] ——, "Teaching Multiple Tasks to an RL Agent using LTL," in *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems*, 2018, pp. 452–461. [Online]. Available: https://www.ifaamas.org/Proceedings/aamas2018/pdfs/p452.pdf

[13] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," in *International Conference on Learning Representations*, 2017. [Online]. Available: https://arxiv.org/abs/1609.02907

[14] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural Message Passing for Quantum Chemistry," in *International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 70. PMLR, 2017, pp. 1263–1272. [Online]. Available: https://proceedings.mlr.press/v70/gilmer17a.html

[15] W. Qiu, W. Mao, and H. Zhu, "Instructing Goal-Conditioned Reinforcement Learning Agents with Temporal Logic Objectives," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 39 147–39 175. [Online]. Available: https://papers.nips.cc/paper_files/paper/2023/hash/7b35a69f434b5eb07ed1b1ef16ace52c-Abstract-Conference.html

[16] V. P. Dwivedi, A. T. Luu, T. Laurent, Y. Bengio, and X. Bresson, "Graph Neural Networks with Learnable Structural and Positional Representations," in *International Conference on Learning Representations*, 2022. [Online]. Available: https://arxiv.org/abs/2110.07875

[17] R. Sato, "A Survey on The Expressive Power of Graph Neural Networks," 2020. [Online]. Available: https://arxiv.org/abs/2003.04078

[18] C. Ying, T. Cai, S. Luo, S. Zheng, G. Ke, D. He, Y. Shen, and T.-Y. Liu, "Do Transformers Really Perform Badly for Graph Representation?" in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 28 877–28 888. [Online]. Available: https://papers.nips.cc/paper_files/paper/2021/hash/f1c1592588411002af340cbaedd6fc33-Abstract.html

[19] C. Morris, M. Ritzert, M. Fey, W. L. Hamilton, J. E. Lenssen, G. Rattan, and M. Grohe, "Weisfeiler and Leman Go Neural: Higher-Order Graph Neural Networks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, 2019, pp. 4602–4609. [Online]. Available: https://doi.org/10.1609/aaai.v33i01.33014602

[20] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive Representation Learning on Large Graphs," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: https://papers.nips.cc/paper/2017/hash/5dd9db5e033da9c6fb5ba83c7a7ebea9-Abstract.html

[21] Y. Li, C. Gu, T. Dullien, O. Vinyals, and P. Kohli, "Graph Matching Networks for Learning the Similarity of Graph Structured Objects," in *Proceedings of the International Conference on Machine Learning*, vol. 97, 2019, pp. 3835–3845. [Online]. Available: https://proceedings.mlr.press/v97/li19d.html

[22] R. S. Sutton, D. Precup, and S. Singh, "Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning," *Artificial Intelligence*, vol. 112, no. 1-2, pp. 181–211, 1999. [Online]. Available: https://doi.org/10.1016/s0004-3702(99)00052-1

[23] P.-L. Bacon, J. Harb, and D. Precup, "The Option-Critic Architecture," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 31, no. 1, 2017. [Online]. Available: https://doi.org/10.1609/aaai.v31i1.10916

[24] O. Nachum, S. S. Gu, H. Lee, and S. Levine, "Data-Efficient Hierarchical Reinforcement Learning," in *Advances in Neural Information Processing Systems*, vol. 31, 2018. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2018/hash/e638471149171 3d29bc63fc5eeb5ba4f-Abstract.html

[25] A. S. Vezhnevets, S. Osindero, T. Schaul, N. Heess, M. Jaderberg, D. Silver, and K. Kavukcuoglu, "FeUdal Networks for Hierarchical Reinforcement Learning," in *Proceedings of the International Conference on Machine Learning*, vol. 70, 2017, pp. 3540–3549. [Online]. Available: https://proceedings.mlr.press/v70/vezhnevets17a.html

[26] H. Hasanbeig, D. Kroening, and A. Abate, "Certified reinforcement learning with logic guidance," *Artificial Intelligence*, vol. 322, p. 103949, 2023. [Online]. Available: https://doi.org/10.1016/j.artint.2023.103949

[27] G. Katz, C. Barrett, D. L. Dill, K. Julian, and M. J. Kochenderfer, "Reluplex: An Efficient SMT Solver for Verifying Deep Neural Networks," in *Computer Aided Verification*, 2017, pp. 97–117. [Online]. Available: https://doi.org/10.1007/978-3-319-63387-9_5

[28] M. Gaon and R. Brafman, "Reinforcement Learning with Non-Markovian Rewards," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 04, 2020, pp. 3980–3987. [Online]. Available: https://doi.org/10.1609/aaai.v34i04.5814

[29] C. Glanois, P. Weng, M. Zimmer, D. Li, T. Yang, J. Hao, and W. Liu, "A survey on interpretable reinforcement learning," *Machine Learning*, vol. 113, no. 8, pp. 5847–5890, 2024. [Online]. Available: https://doi.org/10.1007/s10994-024-06543-w

[30] A. Camacho, R. Toro Icarte, T. Q. Klassen, R. Valenzano, and S. A. McIlraith, "LTL and Beyond: Formal Languages for Reward Function Specification in Reinforcement Learning," in *Proceedings of the International Joint Conference on Artificial Intelligence*, 2019, pp. 6065–6073. [Online]. Available: https://www.ijcai.org/proceedings/2019/840

[31] S. Brody, U. Alon, and E. Yahav, "How Attentive are Graph Attention Networks?" in *International Conference on Learning Representations*, 2022. [Online]. Available: https://arxiv.org/abs/2105.14491

[32] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of*

the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 770–778. [Online]. Available: https://openaccess.thecvf.com/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html

[33] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer Normalization," 2016. [Online]. Available: https://arxiv.org/abs/1607.06450

[34] L. Rampášek, M. Galkin, V. P. Dwivedi, A. T. Luu, G. Wolf, and D. Beaini, "Recipe for a General, Powerful, Scalable Graph Transformer," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 14 501–14 515. [Online]. Available: https://papers.nips.cc/paper_files/paper/2022/hash/5d4834a159f1547b267a05a4e2b7cf5e-Abstract-Conference.html

[35] J. Brandstetter, D. Worrall, and M. Welling, "Message Passing Neural PDE Solvers," in *International Conference on Learning Representations*, 2022. [Online]. Available: https://arxiv.org/abs/2202.03376

[36] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," 2017. [Online]. Available: https://arxiv.org/abs/1707.06347

[37] D. P. Kingma and J. L. Ba, "Adam: A Method for Stochastic Optimization," in *International Conference on Learning Representations*, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980

[38] H. Wang, Z. Jiang, Y. You, Y. Han, G. Liu, J. Srinivasa, R. Kompella, and Z. A. Wang, "Graph Mixture of Experts: Learning on Large-Scale Graphs with Explicit Diversity Modeling," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 50 825–50 837. [Online]. Available: https://papers.nips.cc/paper_files/paper/2023/hash/9f4064d145bad5e361206c3303bda7b8-Abstract-Conference.html

[39] K. P. Murphy, *Machine Learning: A Probabilistic Perspective*. MIT Press, 2012. [Online]. Available: https://www.cs.ubc.ca/~murphyk/MLbook/

[40] V. H. Pong, A. V. Nair, L. M. Smith, C. Huang, and S. Levine, "Offline Meta-Reinforcement Learning with Online Self-Supervision," in *Proceedings of the International Conference on Machine Learning*, vol. 162, 2022, pp. 17 811–17 829. [Online]. Available: https://proceedings.mlr.press/v162/pong22a.html